

■ HomelyHub — Project Submission Report

Full-Stack MERN Vacation Rental & Property Booking Platform 4-Week Internship Comprehensive Documentation

Project: HomelyHub	Architecture: MERN Stack (MongoDB, Express, React, Node)
Frontend: React 18 + Vite (Vercel)	Backend: Express.js REST API (Render)
Database: MongoDB Atlas Cloud	Live URL: https://homely-hub-omega.vercel.app
GitHub: github.com/Shivam95800/HomelyHub	Author / Developer: Shivam

1. Executive Summary

HomelyHub is an end-to-end full-stack property booking platform modeled as a streamlined, modern alternative to platforms like Airbnb. The system allows travelers (guests) to search verified vacation homes, apply multi-criteria filters (by location, keyword, and budget), view real-time property galleries, dynamically calculate total costs based on stay duration, and instantly reserve dates with rigorous double-booking collision protection. Property owners (hosts) can create and manage listings.

2. Technology Stack Specifications

Domain	Technologies Used	Purpose
Frontend UI	React.js 18, Vite 5, React Router DOM	Single Page Application (SPA) with fast client routing
Design & Styling	Custom Dark & Obsidian CSS, Plus Jakarta Sans	Modern Bento-Grid layout, frosted glass, responsive design
Backend API	Node.js (ES Modules), Express.js 4	RESTful API architecture with CORS & JSON body parsing
Database / ODM	MongoDB Atlas Cloud, Mongoose 8	Schema validation, indexing, and multi-collection references
Security & Auth	JSON Web Tokens (JWT), Bcrypt.js	Stateless Bearer token auth and 10-round salted password hashing
HTTP Client	Axios (with Request Interceptors)	Automatic JWT header attachment for protected endpoints
Cloud Hosting	Vercel (Frontend), Render (Backend)	High availability production deployment with CI/CD

3. Database Schema Design (Mongoose)

The application maintains three core models in MongoDB with strict validation and relational references:

- **User Model (models/User.js):** name, email (unique, lowercase), password (hashed with bcrypt), role (enum: 'guest' | 'owner'), timestamps.
- **Property Model (models/Property.js):** title, description, price (per night), location (indexed for search), images (array of URLs), amenities (array of strings), ownerId (ref to User), timestamps.
- **Booking Model (models/Booking.js):** propertyId (ref to Property), userId (ref to User), checkInDate, checkOutDate, totalPrice, status (enum: 'confirmed' | 'cancelled' | 'pending'), timestamps.

4. Key Algorithms & Business Logic

A. Mathematical Date-Overlap Collision Detection Algorithm:

To guarantee that two users can never reserve the same property for overlapping dates, the backend booking controller executes an interval query in MongoDB prior to confirming any reservation. Two date ranges [CheckIn, CheckOut] overlap if and only if:

```
existingCheckIn < newCheckOut AND existingCheckOut > newCheckIn (WHERE status != 'cancelled')
```

If any existing booking satisfies this condition for the target property, the server rejects the request with HTTP 400 ('Property already booked for selected dates').

B. Dynamic Night and Total Cost Calculation:

```
Total Nights = Math.ceil( |checkOutDate - checkInDate| / (1000 * 60 * 60 * 24) )  
Total Price = Total Nights * Property.PricePerNight
```

5. REST API Endpoints Specification

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new guest or owner account with bcrypt hashing
POST	/api/auth/login	Public	Authenticate credentials and return JWT bearer token
GET	/api/auth/me	Private	Get current authenticated user profile details
GET	/api/properties	Public	List all properties with filters (location, search, min/max price)
GET	/api/properties/:id	Public	Get single property details with populated owner info
POST	/api/properties	Owner	Create a new property listing (protected, owner only)
PUT	/api/properties/:id	Owner	Update an existing owned property listing
DELETE	/api/properties/:id	Owner	Delete an owned property listing from database
POST	/api/bookings	Private	Create reservation with collision overlap prevention
GET	/api/bookings/my	Private	Fetch all bookings of logged-in user with populated stay data
PATCH	/api/bookings/:id/cancel	Private	Cancel an active booking reservation
GET	/api/health	Public	Health check & uptime monitor endpoint

6. Frontend Architecture & State Management

- **AuthContext (Global Session Provider):** Manages user state, token in localStorage, login, register, and logout. Prevents state loss on browser refresh.
- **ProtectedRoute Component:** Wraps private pages (/my-bookings), redirecting unauthorized guests to /login with automatic return redirect.
- **Axios Interceptors:** Automatically attaches Bearer tokens to request headers and manages global 401 unauthorized errors.
- **Modern Dark Obsidian UI:** Crafted using responsive CSS variables, glassmorphism frosted headers, and Plus Jakarta Sans typography.

7. Comprehensive Viva & Interview Q&A;

Q1: What are the main benefits of using the MERN stack for this project?

MERN enables end-to-end JavaScript/ES Module development across client and server. It allows seamless JSON object interchange between MongoDB, Express, and React without complex serialization, offering high performance, rapid development, and rich single-page application (SPA) capabilities.

Q2: How does the platform handle password security and authentication?

Passwords are encrypted using Bcrypt.js with 10 salt rounds before persisting in MongoDB. Upon valid authentication, the server issues a signed JSON Web Token (JWT) with a 30-day expiry. The client persists this token and presents it in the 'Authorization: Bearer ' header on all secured routes.

Q3: Explain how the date-overlap booking verification works.

In bookingController.js, we query MongoDB using the interval collision condition: (existingCheckIn < newCheckOut && existingCheckOut > newCheckIn) for active (non-cancelled) bookings on that propertyId. If any matching record exists, the reservation is rejected with an explanatory message.

Q4: How are environment variables handled across development and cloud deployment?

Locally, variables are stored in .env (ignored by .gitignore). In production, variables (PORT, MONGO_URI, JWT_SECRET, CLIENT_URL, VITE_API_BASE_URL) are securely injected via Render and Vercel environment dashboards.